# CONWIP - Constant Work in Process

## Definition

A production control method that maintains a constant number of jobs in the system

**Developed by:** Spearman, Woodruff, and Hopp (1990)

## Key Principle:

- New job enters system only when a job exits
- WIP level is fixed (system-wide constant)
- Combines benefits of push and pull

# CONWIP vs Kanban

| Feature | CONWIP | Kanban |
|---|---|---|
| WIP Control | System-wide | Stage-by-stage |
| Flexibility | High | Lower |
| Cards/Signals | One type | Multiple types |
| Product Mix | Handles variety well | Works best with stable mix |
| Implementation | Simpler | More complex |
| Bottleneck Impact | Self-adjusting | May require rebalancing |

# CONWIP Flow Diagram

# CONWIP Card Calculation

## Little's Law Application

$$WIP = Throughput \times LeadTime$$

### CONWIP Level Determination:

- Target throughput: 20 units/day
- Average lead time: 2 days
- CONWIP level = 20 × 2 = 40 units

**Tuning CONWIP**

Start conservative, measure performance, adjust based on data

# CONWIP Database Implementation

```
CREATE TABLE ConwipSystem ( system_id INTEGER PRIMARY KEY, max_wip INTEGER NOT NULL, current_wip INTEGER DEFAULT 0, available_cards INTEGER);CREATE
TABLE ConwipCards ( card_id INTEGER PRIMARY KEY, system_id INTEGER, status TEXT CHECK(status IN ('Available','Attached','InUse')), attached_to_job
INTEGER, issue_time DATETIME, return_time DATETIME);CREATE TRIGGER update_wip_on_completionAFTER UPDATE ON JobsWHEN NEW.status = 'Completed'BEGIN
UPDATE ConwipSystem SET current_wip = current_wip - 1, available_cards = available_cards + 1 WHERE system_id = NEW.system_id;
```

# CONWIP Advantages

**Benefits:**

- Simpler than multi-stage kanban
- Handles product mix variability well
- Automatically adjusts to bottleneck shifts
- Limits WIP without detailed stage analysis
- Easy to implement and understand
- Provides predictable lead times
- Robust to process variability

**Research Finding**

CONWIP achieves 90-95% of kanban performance with 50% less complexity

# CONWIP Disadvantages

**Limitations:**

- Less stage-by-stage control than kanban
- May not minimize inventory at each stage
- Requires stable routing structure
- Can accumulate WIP at slowest stage
- Less effective with highly variable routings
- Doesn't inherently balance stages

# Production System Comparison Matrix

| Criterion | Push (MRP) | Pull (Kanban) | JIT | CONWIP |
|---|---|---|---|---|
| WIP Control | Indirect | Direct (stage) | Direct (system) | Direct (system) |
| Flexibility | High | Low | Medium | High |
| Lead Time | Long | Short | Very Short | Medium |
| Complexity | High | Medium | High | Low |
| Demand Variability | Handles well | Struggles | Struggles | Handles well |
| Product Variety | High | Low | Low | Medium-High |
| Setup Time | Any | Must be low | Very low | Medium |
| Quality Req. | Medium | High | Very High | High |

# Hybrid Production Control Systems

## Real-World Reality

Most successful manufacturing systems use hybrid approaches

### Common Hybrids:

- MRP for long-term planning + Kanban for shop floor
- TOC for constraint management + JIT for other areas
- Push for raw materials + Pull for finished goods
- CONWIP for high-variety products + Kanban for runners

**MES Role:** Support multiple control methods simultaneously

# Discussion: Which System for Which Environment?

## Scenario Analysis

For each scenario, which production control system would you recommend?

01

**High-mix, low-volume custom electronics assembly**

02

**Automotive final assembly line (stable demand)**

03

**Aerospace component manufacturing (long lead times)**

04

**Food processing (short shelf life, variable demand)**

Discuss in small groups: 5 minutes

# Why Databases Matter for Production Control

- Store and track production state for any control system
- Enable real-time visibility into WIP and status
- Support decision-making with historical data
- Provide audit trails for compliance
- Enable integration between systems
- Calculate metrics and KPIs

**Critical Point**

Your database schema must support your production control philosophy

# Relational Database Core Concepts

## Tables (Relations)

- Organized in rows and columns
- Each row is a record
- Each column is an attribute

## Relationships

- One-to-many
- Many-to-many
- One-to-one

## Primary and Foreign Keys

**Primary Key (PK)**

Unique identifier for each record in a table

**Foreign Key (FK)**

Column that references a primary key in another table

**Example:**

- Orders table has order_id (PK)
- Operations table has order_id (FK) referencing Orders

- This creates a one-to-many relationship

Keys ensure data integrity and enable joins

# Example: Core MES Tables

```
CREATE TABLE Products ( product_id INTEGER PRIMARY KEY, product_number TEXT UNIQUE NOT NULL, description TEXT, standard_cost REAL);CREATE TABLE
WorkCenters ( workcenter_id INTEGER PRIMARY KEY, workcenter_code TEXT UNIQUE NOT NULL, capacity_per_hour REAL, efficiency REAL DEFAULT 1.0);CREATE
TABLE Routings ( routing_id INTEGER PRIMARY KEY, product_id INTEGER, operation_seq INTEGER, workcenter_id INTEGER, setup_time_minutes REAL,
run_time_per_unit REAL, FOREIGN KEY(product_id) REFERENCES Products(product_id), FOREIGN KEY(workcenter_id) REFERENCES WorkCenters(workcenter_id));
```

# Normalization Principles

## Database Normalization

Process of organizing data to reduce redundancy and improve integrity

## Normal Forms:

**1NF:** Atomic values (no repeating groups)

**2NF:** No partial dependencies

**3NF:** No transitive dependencies

**BCNF:** Every determinant is a candidate key

**Goal:** Each table represents one concept; minimize data duplication

# Push System Queries

```
-- Find orders to release based on planned startSELECT o.order_id, o.product_id, o.quantity, o.planned_start,
o.due_dateFROM Orders oWHERE o.planned_start <= date('now') AND o.status = 'Planned'ORDER BY o.priority DESC,
o.due_date ASC;-- Calculate MRP requirementsSELECT p.product_id, SUM(o.quantity) AS total_demand, p.on_hand_quantity,
(SUM(o.quantity) - p.on_hand_quantity) AS net_requirementFROM Orders oJOIN Products p ON o.product_id =
p.product_idWHERE o.due_date BETWEEN date('now') AND date('now', '+30 days')GROUP BY p.product_idHAVING
net_requirement > 0;
```

# Pull System Queries

```
-- Check for available kanban cardsSELECT k.kanban_id, k.part_id, k.quantityFROM KanbanCards kWHERE k.status = 'Available'
AND k.to_location = 'Assembly'ORDER BY k.part_id;-- Trigger replenishment based on consumptionSELECT k.part_id, COUNT(*) AS
cards_in_use, MAX(k.quantity) AS replenish_qtyFROM KanbanCards kWHERE k.status = 'InUse' AND k.last_movement <
datetime('now', '-1 hour')GROUP BY k.part_idHAVING cards_in_use >= ( SELECT kanban_count * 0.8 FROM KanbanSettings WHERE
part_id = k.part_id);
```

# TOC Constraint Identification

```
-- Identify bottleneck by utilizationSELECT w.workcenter_code, COUNT(o.operation_id) AS queue_depth,
AVG(o.wait_time_minutes) AS avg_wait, SUM(o.processing_time) / w.available_time * 100 AS utilization_pctFROM Operations
oJOIN WorkCenters w ON o.workcenter_id = w.workcenter_idWHERE o.status IN ('Waiting', 'Processing') AND o.scheduled_date
= date('now')GROUP BY w.workcenter_idORDER BY utilization_pct DESC, queue_depth DESCLIMIT 1;-- Buffer status for
constraintSELECT COUNT(*) AS buffer_size, SUM(CASE WHEN priority='High' THEN 1 ELSE 0 END) AS urgent_jobsFROM
OperationsWHERE workcenter_id = (SELECT constraint_wc_id FROM TOCSettings) AND status = 'Waiting';
```

# CONWIP Monitoring

```
-- Check CONWIP levelSELECT c.system_name, c.max_wip, COUNT(j.job_id) AS current_wip, c.max_wip - COUNT(j.job_id) AS
available_capacityFROM ConwipSystem cLEFT JOIN Jobs j ON c.system_id = j.system_id AND j.status IN ('Released',
'InProcess')GROUP BY c.system_id;-- Find jobs ready to release (if capacity available)SELECT j.job_id, j.product_id,
j.priorityFROM Jobs jWHERE j.status = 'Waiting' AND (SELECT current_wip FROM ConwipSystem WHERE system_id =
j.system_id) < (SELECT max_wip FROM ConwipSystem WHERE system_id = j.system_id)ORDER BY j.priority DESC, j.due_date
ASC;
```

# Assignment 1: Production System Analysis

## Objective

Analyze a real or hypothetical manufacturing environment

## Tasks:

1. Describe the manufacturing process (5-10 workstations)
2. Identify current production control method
3. Map to ISA-95 levels
4. Analyze strengths and weaknesses
5. Recommend alternative approaches

**Deliverable:** 3-5 page report with diagrams

**Due:** One week from today

# Assignment 2: Database Schema Design

## Objective

Design and implement an MES database for a specific production control system

### Requirements:

1. Choose ONE production philosophy (Push, Pull, JIT, TOC, or CONWIP)
2. Design tables to support that philosophy
3. Include minimum 8 tables
4. Define all primary and foreign keys
5. Write 5 queries that support the chosen system
6. Populate with sample data (at least 20 records per table)

**Deliverable:** SQLite database file + documentation

**Due:** Two weeks from today